



Examen MU4IN505 CPA. Durée: 2h.

BM. Bui-Xuan

Tout document personnel non-électronique autorisé. Toute réponse sans preuve aura valeur nulle.

Exercice 1. Estimation du temps de calcul (2 points)

Supposons que nous avons à exécuter sur un ordinateur un processus prenant en entrée un argument de taille $\approx n$ B (byte). Supposons également que l'unité de calcul (CPU) de cet ordinateur effectue ≈ 1 G (giga) instructions par seconde. Nous assumons que 1B est la plus petite unité accessible par ordinateur à chaque instruction.

Q1. L'analyse théorique de ce processus prouve que sa complexité est de l'ordre de $\Theta(n^3)$. Estimer le temps de calcul de ce processus pour:

- $n = 1$ K (kilo); $n = 10000$ (dix mille); $n = 1$ M (mega); $n = 1$ G (giga); $n = 1$ T (tera).

Q2. Donner les mêmes (cinq) estimations en supposant que la complexité théorique du processus est en $\Theta(n \log^2 n)$.

Exercice 2. Enveloppe convexe (4 points)

D'un point de vue de structure de données en 2D, un *polygone* est une liste ordonnée de points dans le plan, où l'ordre des points suit le sens trigonométrique du repère cartésien (c.à.d. de l'axe $\vec{Ox}$ vers l'axe $\vec{Oy}$). Avec cette structure de donnée, on appelle *côté* du polygone tout segment reliant deux points consécutifs dans la liste définissant le polygone. Un polygone est *simple* si la seule intersection possible entre deux côtés quelconques du polygone est l'une des extrémités de ces deux côtés.

Par abus de langage, on confond le polygone et la partie du plan située à l'intérieur des segments délimitant le polygone. Un polygone est *convexe* s'il est simple et s'il contient tout segment reliant deux points à l'intérieur du polygone. L'*enveloppe convexe* d'un objet géométrique (liste de points, liste de segments, polygone, ...) est le plus petit polygone convexe contenant cet objet. Dans les questions suivantes, les seuls langages de programmation autorisés sont: java, python et C.

Q1. (Question de cours) Montrer que la complexité optimale au pire cas d'un algorithme calculant l'enveloppe convexe d'un ensemble de n points dans le plan est $\Theta(n \log n)$.

Rappel important: pour qu'une réponse soit considérée, il est impératif de la justifier...

Exercice 3. Chemin (2 points)

Nous considérons la représentation d'un graphe pondéré G par une matrice M_G : $M_G(i, j)$ a pour valeur ∞ s'il n'y a pas d'arête entre les sommets i et j ; $M_G(i, j)$ a pour valeur le poids de l'arête dans le cas contraire. Nous considérons de plus que le poids de l'arête représente le coût pour traverser l'arête (i, j) .

Q1. Une méthode naïve pour calculer la matrice de distance M_{dist} , où $M_{dist}(i, j)$ a pour valeur la distance géodésique (i.e. celle dans le graphe G) entre les sommets i et j est de définir M^p , la matrice où $M^p(i, j)$ est la distance entre i et j des chemins utilisant au plus p sommets dans G , pour $2 \leq p \leq n$ et n le nombre de sommets dans G . Nous remarquons la formule de récurrence suivante:

$$M^{p+1}(i, j) = \min_k M^p(i, k) + M^2(k, j).$$

Proposer un algorithme calculant M_{dist} en passant par le calcul de M^p , pour $2 \leq p \leq n$. Quelle est sa complexité?

Q2. Soit N^p la matrice où $N^p(i, j)$ est la distance entre i et j des chemins passant uniquement par les p premiers sommets de G . Donner une formule de récurrence pour le calcul de N^{p+1} à partir de la connaissance de N^p . En déduire un calcul plus rapide de M_{dist} en passant par le calcul de N^p . Quelle est sa complexité?

Exercice 4. Chemin temporel (1 points)

Un *flot de liens* (ou graphe évolutif, ou graphe dynamique, ou graphe temporel) est un triplet $L = (T, V, E)$ où T est un intervalle d'entiers, V un ensemble fini d'éléments appelés sommets, et E un ensemble vérifiant

$$E \subseteq \{(\{u, v\}, t) : t \in T \wedge u \in V \wedge v \in V \wedge u \neq v\}.$$

Les éléments de E sont appelés des arêtes temporelles. Pour simplifier l'écriture, quand (et seulement quand) $u \neq v$, on note aussi uv l'ensemble $\{u, v\}$. Ainsi, on peut noter

$$E \subseteq \{(uv, t) : t \in T \wedge u \in V \wedge v \in V \wedge u \neq v\}.$$

Soit L_{ex} le flot de liens défini par $L_{ex} = (T_{ex}, V_{ex}, E_{ex})$ où:

- $T_{ex} = \llbracket 1, 7 \rrbracket$
- $V_{ex} = \{a, b, c, d, e, f, g\}$
- $E_{ex} = \{(ab, 1), (bc, 1), (bd, 1), (ab, 2), (cd, 2), (ab, 3), (de, 3), (ab, 4), (bf, 4), (ab, 5), (bf, 5), (fg, 5), (ab, 6), (bf, 6), (fg, 6), (ag, 6), (ab, 7), (bf, 7), (fg, 7), (ag, 7)\}$

Une représentation graphique du flot de liens L_{ex} est donnée dans la Figure 1.

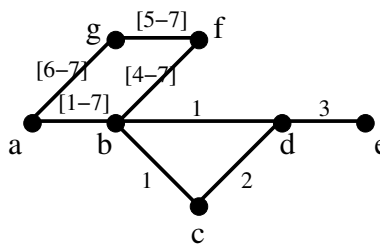


Figure 1: Flot de liens L_{ex} .

Un *chemin temporel* de $u \in V$ vers $v \in V$ arrivant à la date $t \in T$ est une suite d'arêtes temporelles $(u_0 u_1, t_1), (u_1 u_2, t_2), \dots, (u_{k-1} u_k, t_k)$ telle que: $u = u_0$, $v = u_k$, $t = t_k$, et telle que $t_1 < t_2 < \dots < t_k$ et $(u_{i-1} u_i, t_i) \in E$ pour tout $1 \leq i \leq k$. Un *plus court*¹ *chemin temporel* de u vers v est formellement défini comme un chemin temporel de u vers v arrivant à une date $t \in T$ telle que t est minimum.

Par exemple, dans le flot de liens L_{ex} on a (au moins) deux chemins temporels de a vers f arrivant à la date 7:

¹Ici, la notion de distance est donnée par le fait d'être "arrivé au plus tôt". En réalité, il existe au moins deux autres notions de distance d'un chemin temporel, qui ne seront pas abordées pendant l'examen.

- premier chemin: $(ab, 1), (bf, 7)$
- deuxième chemin: $(ag, 6), (fg, 7)$

En revanche, ni l'un ni l'autre est un plus court chemin temporel, parce qu'il existe un chemin temporel de a vers f arrivant à la date 4.

Dans le flot de liens L_{ex} :

- Q1.** donner un plus court chemin temporel de b vers e . Existe-t-il un autre plus court chemin temporel de b vers e ?
- Q2.** donner un plus court chemin temporel de b vers d . Existe-t-il un autre plus court chemin temporel de b vers d ?
- Q3.** pour tout sommet $v \in V_{ex}$, donner un plus court chemin temporel de b vers v .

Exercice 5. Chemin temporel plus court de toutes parts (5 points)

Soit $L = (T, V, E)$ un flot de liens. Soit $C = (u_0u_1, t_1), (u_1u_2, t_2), \dots, (u_{k-1}u_k, t_k)$ un chemin temporel dans L . On appelle préfixe de C tout chemin $(u_0u_1, t_1), (u_1u_2, t_2), \dots, (u_{i-1}u_i, t_i)$ tel que $1 \leq i \leq k$. Un chemin temporel est un *plus court chemin temporel de toutes parts* si tout préfixe de ce chemin est lui-même un plus court chemin temporel.

- Q1.** Dans le flot de liens L_{ex} , existe-t-il un plus court chemin temporel de toutes parts?
- Q2.** Dans le flot de liens L_{ex} , existe-t-il un plus court chemin temporel qui n'est pas un plus court chemin temporel de toutes parts?
- Q3.** Rappeler la définition d'un graphe (classique). Rappeler la définition d'un chemin dans un graphe. Rappeler la définition d'un plus court chemin dans un graphe. Donner un exemple de graphe avec au plus cinq sommets. Dans cet exemple, existe-t-il un plus court chemin qui n'est pas un plus court chemin de toutes parts?
- Q4.** Donner un algorithme calculant un plus court chemin temporel de toutes parts de u à v , pour tout $u, v \in V$.

N.B.: la définition des chemins temporels plus courts de toutes parts est donnée plus haut dans le texte.

Pour l'exemple en question, voici les résultats:

- oui: $(bd, 1), (de, 3)$ dont le préfixe $(bd, 1)$ est également un plus court chemin temporel.
- oui: $(bc, 1), (cd, 2), (de, 3)$ dont le préfixe $(bc, 1), (cd, 2)$ n'est pas un plus court chemin temporel.
- Un graphe est un couple (V, E) où V est un ensemble fini d'éléments appelés sommets, et E un ensemble vérifiant

$$E \subseteq \{\{u, v\} : u \in V \wedge v \in V \wedge u \neq v\}.$$

Les éléments de E sont appelés des arêtes.

- Un chemin de $u \in V$ vers $v \in V$ est une suite d'arêtes $(u_0u_1), (u_1u_2), \dots, (u_{k-1}u_k)$ telle que: $u = u_0, v = u_k$, et telle que $(u_{i-1}u_i) \in E$ pour tout $1 \leq i \leq k$.
- Un exemple de graphe est $G_{ex} = (V_{ex}, E_{ex})$ où:
 - $V_{ex} = \{a, b, c, d, e\}$

- $E_{ex} = \{ab, bc, bd, de\}$
- Dans le graphe G_{ex} tout chemin plus court est aussi plus court de toutes parts. Exemple: bd, de . En particulier, bc, cd, de n'est pas un plus court chemin. Ce cas se distingue du cas temporel plus haut où $(bc, 1), (cd, 2), (de, 3)$ est un plus court chemin temporel...
- La propriété d'être "plus court de toutes parts" où tout préfixe d'un chemin optimal est aussi un chemin optimal nous permet de penser à l'approche glouton. Plus précisément, un algorithme pour calculer un chemin temporel plus court de toutes parts peut s'inspirer de l'approche glouton utilisée dans l'algorithme Dijkstra. Nous devons ici prendre comme poids des arêtes l'instant minimal t où l'arête est disponible. Voici le pseudo-code pour calculer un chemin temporel plus courts de toutes parts d'un sommet source $s \in V$ vers n'importe quel autre sommet $v \in V$:
 - pour tout $v \neq u$ dans V :
 - * $distance[v] = \infty$
 - * $color[v] = GREEN$
 - $distance[s] = 0$
 - $color[s] = RED$
 - $Q = \{s\}$
 - tant que Q n'est pas vide:
 - * enlever l'élément u dans Q dont $distance[u]$ est minimum
 - * $color[u] = RED$
 - * pour tout v tel que $color[v] == GREEN$ et il existe une arête temporelle entre u et v :
 - $t = \text{l'instant minimum } t > distance[u] \text{ tel que l'arête } uv \text{ est disponible}$
 - si $t < distance[v]$, mettre à jour $distance[v] = t$ et si de plus $v \notin Q$, ajouter v dans Q
- Pour obtenir les derniers points bonus, il reste encore à prouver que l'algorithme calcule correctement un chemin temporel plus court de toutes parts.

Exercice 6. Composante connexe (6 points)

Soit $L = (T, V, E)$ un flot de liens. Une *partie connexe* A est un sous-ensemble de sommets $A \subseteq V$ tel que pour tout $u \in A$ et $v \in A$, il existe un chemin temporel de u vers v (et il existe un chemin temporel de v vers u). Une *composante connexe* est une partie connexe maximale par inclusion². Une *composante connexe maximum* est une composante connexe de taille maximum.

- Q1.** Dans l'exemple L_{ex} , s'elle existe, donner: une partie connexe A telle que $|A| > 2$; une composante connexe; et une composante connexe maximum.
- Q2.** On considère maintenant le cas général. Si le problème est polynomial, donner un algorithme calculant une partie connexe A telle que $|A| > 2$.
- Q3.** Si le problème est polynomial, donner un algorithme calculant une composante connexe.
- Q4.** Si le problème est polynomial, donner un algorithme calculant une composante connexe maximum.

²En d'autres termes, une partie connexe A est appelée composante connexe s'il n'existe pas de partie connexe B telle que $A \subsetneq B$.